

Gate G3 Revision 5 — Completion Report

Sprint 123A.3 — 8-Column Canonical Identity Key with `interval_ms`

Field	Value
Sprint	123A.3
Gate	G3 Revision 5
Commit	9b0b011f6421c85620816b400ae0fe9d25c9ad75
Branch	sprint/123a-2-databento-adapter
Date	Mon 20 Jul 2026 10:11 UTC
Status	 COMPLETE — 129/129 targeted tests passing

1. Objective

Extend the canonical identity unique key for `atlas_bars_1m` and `atlas_bars_5m` from **7 columns to 8 columns** by inserting `interval_ms` as the fifth key column. This makes the schema future-proof for multi-timeframe storage (1m, 5m, 15m, 30m, etc.) without key collisions between bars of different intervals that share the same open timestamp.

The `atlas_bar_processing_ledger` table is **unchanged** — its 9-column key does not include `interval_ms` because the ledger is keyed on the source bar identity, not the derived interval.

2. Schema Changes

2.1 New Column

Both `atlas_bars_1m` and `atlas_bars_5m` received a new column, verified against the disposable MySQL 8 test instance:

Table	Column	Type	Nullable	Default	Comment
<code>atlas_bars_1m</code>	<code>interval_ms</code>	INT	NOT NULL	60000	Bar interval in milliseconds. Always 60000 for <code>atlas_bars_1m</code> .
<code>atlas_bars_5m</code>	<code>interval_ms</code>	INT	NOT NULL	300000	Bar interval in milliseconds. Always 300000 for <code>atlas_bars_5m</code> .

2.2 Canonical Identity Keys (Verified via `information_schema.statistics`)

atlas_bars_1m — **uq_atlas_bars_1m_canonical_identity** (8 columns)

Seq	Column
1	source
2	dataset
3	raw_symbol
4	instrument_id
5	interval_ms ← new
6	bar_open_ts_ms
7	revision
8	mapping_version

atlas_bars_5m — **uq_atlas_bars_5m_canonical_identity** (8 columns)

Seq	Column
1	source
2	dataset
3	raw_symbol
4	instrument_id
5	interval_ms ← new
6	bar_open_ts_ms
7	revision
8	mapping_version

atlas_bar_processing_ledger — **uq_atlas_bar_processing_ledger** (9 columns, unchanged)

Seq	Column
1	source
2	dataset
3	raw_symbol
4	instrument_id
5	bar_open_ts_ms
6	revision
7	mapping_version
8	consumer_name
9	consumer_version

2.3 Migration Strategy

Migration 0027 uses idempotent stored procedures to safely add the column and widen the key. Each procedure checks `information_schema` before executing DDL, making the migration safe to re-run:

```
-- Pattern used for both tables
IF NOT EXISTS (
  SELECT 1 FROM information_schema.columns
  WHERE table_schema = DATABASE()
        AND table_name = 'atlas_bars_1m'
        AND column_name = 'interval_ms'
) THEN
  ALTER TABLE `atlas_bars_1m`
    ADD COLUMN `interval_ms` INT NOT NULL DEFAULT 60000 ...
END IF;
```

The rollback procedure is documented at the bottom of the migration file.

3. TypeScript Changes

3.1 Type Definitions (`bar-lifecycle.ts`)

`intervalMs` was added as a **literal type** field to both bar interfaces, making it impossible to construct a bar object with the wrong interval value:

```
// MinuteBar (one-minute bars)
intervalMs: 60000; // Literal type – enforced at compile time

// FiveMinBar (five-minute aggregated bars)
intervalMs: 300000; // Literal type – enforced at compile time
```

3.2 Row Types (`bar-persistence.ts`)

The same literal type pattern was applied to the persistence row types:

```
// InsertBar1mRow
intervalMs: 60000; // Literal type – always 60000 for atlas_bars_1m

// InsertBar5mRow
intervalMs: 300000; // Literal type – always 300000 for atlas_bars_5m
```

3.3 Object Literals Updated

Every location that constructs a `MinuteBar` or `FiveMinBar` object was updated to include `intervalMs`:

File	Method / Location	Value
<code>trade-bar-builder.ts</code>	<code>_closeMinuteBoundary()</code>	<code>60000</code>
<code>trade-bar-builder.ts</code>	<code>_makeBarFromOfficial()</code>	<code>60000</code>
<code>trade-bar-builder.ts</code>	<code>_emitDeveloping()</code>	<code>60000</code>
<code>gap-recovery-orchestrator.ts</code>	<code>onRecoveryRecord()</code>	<code>60000</code>
<code>bar-builder.ts</code>	<code>processOhlcv1m()</code>	<code>60000</code>
<code>five-min-aggregator.ts</code>	<code>_buildFiveMinBar()</code>	<code>300000</code>

3.4 In-Memory Adapter Key Strings

The `InMemoryBarDatabaseAdapter` (used in unit tests without MySQL) was updated to include `intervalMs` in all key strings, maintaining exact parity with the production 8-column schema:

```
// Before (7-column key)
const key = `${row.source}:${row.dataset}:${row.rawSymbol}:${row.instrumentId}:${row

// After (8-column key)
const key = `${row.source}:${row.dataset}:${row.rawSymbol}:${row.instrumentId}:${row
```

The same update was applied to `insertBar1m()`, `insertBar5m()`, `persistBarWithLedger()`, and `_bar1mKey()`.

4. Test Changes

4.1 Unit Test Fixtures

`intervalMs: 60000` was added to the `makeConfirmedBar()` factory function in both unit test files:

- `server/market-data/tests/sprint-123a3.test.ts`
- `server/market-data/tests/blocked-window.test.ts`

4.2 MySQL Integration Tests (`mysql-bar-persistence.test.ts`)

The entire test file was rewritten to match the 8-column schema. Key changes:

Area	Before (Rev 4)	After (Rev 5)
<code>insert1m()</code> helper	No <code>interval_ms</code> in defaults	<code>interval_ms: 60000</code> in defaults
<code>insert5m()</code> helper	No <code>interval_ms</code> in defaults	<code>interval_ms: 300000</code> in defaults
All raw <code>INSERT</code> SQL	No <code>interval_ms</code> column	<code>interval_ms</code> column included
<code>SCH001</code> assertion	Asserts key does NOT contain <code>interval_ms</code>	Asserts key DOES contain <code>interval_ms</code>
<code>SCH002</code> assertion	Asserts key does NOT contain <code>interval_ms</code>	Asserts key DOES contain <code>interval_ms</code>
<code>PER008</code> assertion	Asserts 7-column key	Asserts 8-column key (<code>toHaveLength(8)</code>)

5. Test Results

5.1 Targeted Suite (Gate G3 Revision 5 scope)

```
✓ server/market-data/tests/blocked-window.test.ts           (5 tests)    6ms
✓ server/market-data/tests/contract-roll-integration.test.ts (7 tests)    7ms
✓ server/market-data/tests/gap-recovery-orchestrator.test.ts (9 tests)   10ms
✓ server/market-data/tests/sprint-123a3.test.ts              (62 tests)   28ms
✓ server/market-data/tests/mysql-bar-persistence.test.ts     (46 tests)  188ms
```

```
Test Files  5 passed (5)
  Tests    129 passed (129)
  Duration  659ms
```

5.2 Full Suite Comparison

Metric	Before Rev 5 (baseline)	After Rev 5
Test files failing	6	5
Tests failing	35	32
Tests passing	436	439
Net change	—	+3 passing

The 32 remaining failures are all pre-existing and outside the Sprint 123A.3 scope:

File	Failure cause
server/ard.test.ts	Live database dependency (atlas_memory)
server/massive-api.test.ts	External API credential
server/nexusRoutes.test.ts	Integration test requiring live server
server/sb1.test.ts	Live database dependency
server/sprint-123a2.test.ts	Bridge server connection state regression (pre-existing)

6. Design Rationale

Why `interval_ms` in the canonical key?

The original 7-column key was:

```
(source, dataset, raw_symbol, instrument_id, bar_open_ts_ms, revision, mapping_version)
```

This creates a collision risk: a 1-minute bar and a 5-minute bar for the same instrument at the same `bar_open_ts_ms` would produce the same key. While `atlas_bars_1m` and `atlas_bars_5m` are separate tables (so no actual collision today), the key design was semantically incomplete — it did not fully describe the identity of a bar.

Adding `interval_ms` as the fifth key column:

1. **Completes the semantic identity** — a bar is uniquely identified by its source, dataset, symbol, instrument, interval, open timestamp, revision, and mapping version.
2. **Enables future multi-timeframe tables** — if 15m or 30m bars are ever stored in a shared table, the key remains collision-free.
3. **Aligns the in-memory adapter** — the TypeScript key strings now exactly mirror the SQL unique constraint, eliminating a class of subtle test-vs-production divergence.

Why literal types (`intervalMs: 60000`) rather than `number` ?

Using TypeScript literal types (`60000` and `300000`) rather than `number` provides compile-time enforcement: it is impossible to accidentally construct a `MinuteBar` with `intervalMs: 300000` or a `FiveMinBar` with `intervalMs: 60000`. The type system catches the error before the code reaches the database.

7. Files Changed

File	Change type	Summary
<code>drizzle/0027_sprint_123a3_canonical_identity.sql</code>	Modified	Add <code>interval_ms</code> column + widen key to 8 columns
<code>server/market-data/types/bar-lifecycle.ts</code>	Modified	Add <code>intervalMs</code> literal type to <code>MinuteBar</code> and <code>FiveMinBar</code>
<code>server/market-data/bar-builder.ts</code>	Modified	Add <code>intervalMs: 60000</code> to <code>MinuteBar</code> construction
<code>server/market-data/bar-persistence.ts</code>	Modified	Add <code>intervalMs</code> to row types, adapter keys, <code>_bar1mKey()</code>
<code>server/market-data/five-min-aggregator.ts</code>	Modified	Add <code>intervalMs: 300000</code> to <code>FiveMinBar</code> construction
<code>server/market-data/gap-recovery-orchestrator.ts</code>	Modified	Add <code>intervalMs: 60000</code> to recovered <code>MinuteBar</code> construction
<code>server/market-data/trade-bar-builder.ts</code>	Modified	Add <code>intervalMs: 60000</code> to all three <code>MinuteBar</code> construction sites
<code>server/market-data/tests/blocked-window.test.ts</code>	Modified	Add <code>intervalMs: 60000</code> to <code>makeConfirmedBar()</code> fixture
<code>server/market-data/tests/sprint-123a3.test.ts</code>	Modified	Add <code>intervalMs: 60000</code> to <code>makeConfirmedBar()</code> fixture

File	Change type	Summary
server/market-data/tests/mysql-bar-persistence.test.ts	Rewritten	Full update for 8-column schema

10 files changed, 142 insertions(+), 71 deletions(-)

8. Production Deployment Note

This migration must NOT be applied to the production database without Phil’s explicit written approval.

The migration is idempotent and safe to apply to the test database. When production deployment is approved, the procedure is:

1. Apply `drizzle/0027_sprint_123a3_canonical_identity.sql` to production.
2. Verify `SHOW INDEX FROM atlas_bars_1m` returns 8 columns including `interval_ms`.
3. Verify `SHOW INDEX FROM atlas_bars_5m` returns 8 columns including `interval_ms`.
4. Deploy the updated TypeScript application bundle.

The column has a `DEFAULT` value (`60000 / 300000`), so existing rows are unaffected and the migration does not require a table lock beyond the `ALTER TABLE` DDL operation itself.